

InternNova Data Analytics Internship

Week 6 Assignment: Git, GitHub & Final Data Analytics Project

Submitted by: Brojo Mohan Dutta

Duration: 1 Week (7 Days) | **Total Marks:** 100

Project overview

Dataset description

Data cleaning process

EDA

Visualizations

Power BI dashboard screenshots

Business insights

Recommendations

Task 1 & 2: Git & GitHub Basics + Git Workflow Practice

Objective: Set up version control for the capstone project and demonstrate a proper Git workflow with meaningful, staged commits.

Explanation: A local Git repository was initialized and linked to a remote GitHub repository, populated across three distinct commits representing the natural stages of the project — raw data, cleaned data, and final deliverables.

Code (Terminal Commands):

Bash

Code (Terminal Commands):

```
# Clone the GitHub repository
git clone https://github.com/brojomohan58-boop/Retail-Sales-Analysis.git

# Enter the repository
cd Retail-Sales-Analysis

# Check repository status
git status

# Stage the Week 6 project files
git add .

# Commit the initial project files
git commit -m "Add initial Week 6 project files"

# Push the initial commit to GitHub
git push

# Pull the latest changes from GitHub
git pull

# Stage documentation changes
git add README.md

# Commit the documentation update
git commit -m "Update project documentation"

# Push the documentation update
git push

# Stage final documentation/report update
git add .

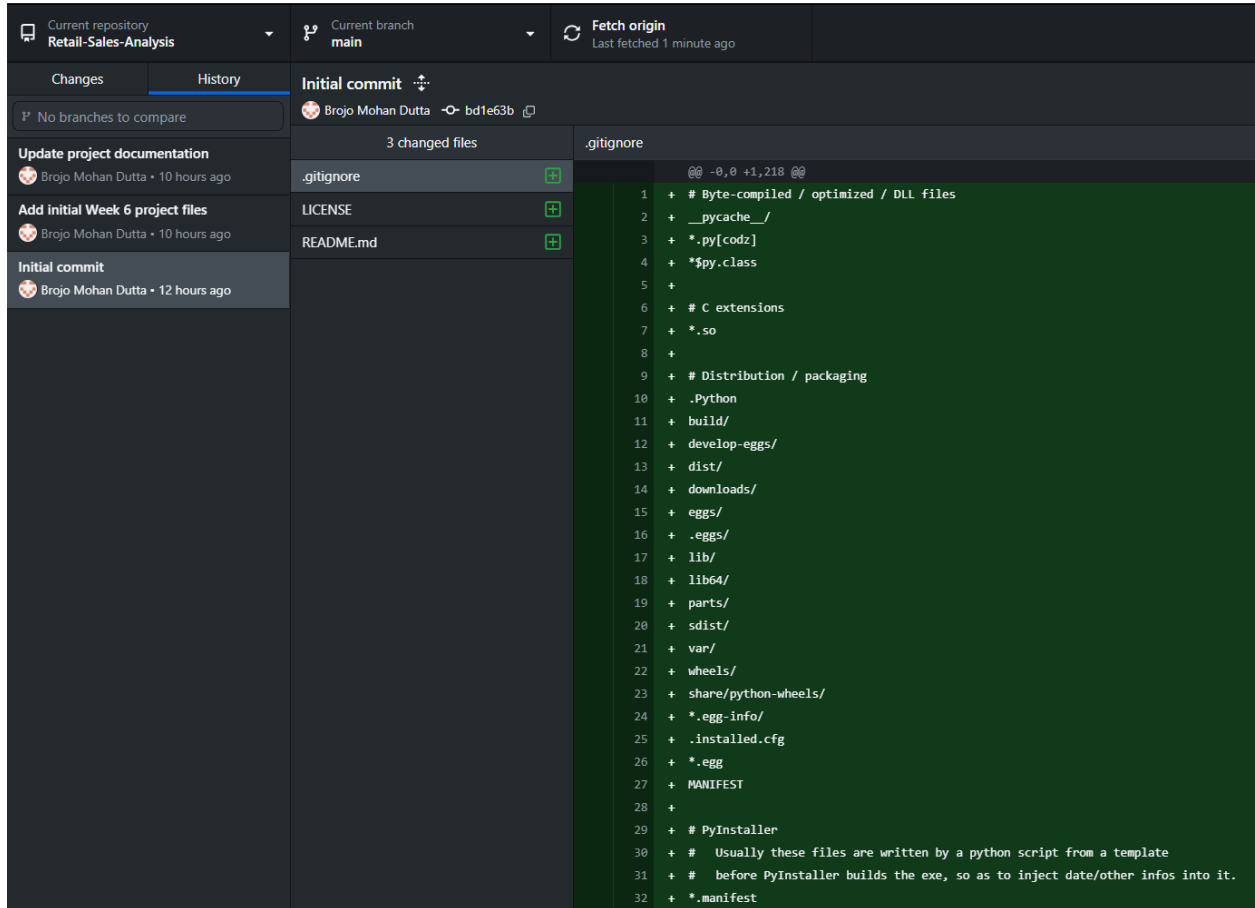
# Create the third meaningful commit
git commit -m "Update final project documentation"

# Push the third commit to GitHub
git push
```

Commit History (3 meaningful commits):

1. "Add initial Week 6 project files"
2. "Update project documentation"
3. "Update final project documentation"

GitHub Repository Link: Retail-Sales-Analysis [Here](#)



The screenshot displays the GitHub web interface for the 'Retail-Sales-Analysis' repository. The top bar shows the current repository, branch (main), and a 'Fetch origin' button. The left sidebar contains a 'Changes' tab and a 'History' tab. The 'History' tab is active, showing a list of commits. The commit history shows three commits: 'Initial commit' (12 hours ago), 'Add initial Week 6 project files' (10 hours ago), and 'Update project documentation' (10 hours ago). The 'Initial commit' is selected, showing a diff view. The diff view shows three changed files: '.gitignore', 'LICENSE', and 'README.md'. The '.gitignore' file is expanded, showing a list of ignored files and directories, including: # Byte-compiled / optimized / DLL files, __pycache__, *.py[cod], *.py.class, # C extensions, *.so, # Distribution / packaging, .Python, build/, develop-eggs/, dist/, downloads/, eggs/, .eggs/, lib/, lib64/, parts/, sdist/, var/, wheels/, share/python-wheels/, *.egg-info/, .installed.cfg, *.egg, MANIFEST, # PyInstaller, and # Usually these files are written by a python script from a template before PyInstaller builds the exe, so as to inject date/other infos into it. *.manifest.

Key Findings: The Git workflow successfully maintained version history for the project through three meaningful commits. The repository was cloned locally, project files were staged and committed, changes were pushed to GitHub, and the git pull command was used to synchronize the local repository with the remote repository.

Task 3: Dataset Selection & Data Preparation

Objective: Select a real-world dataset and prepare it for reliable analysis by resolving missing values, incorrect entries, and structural issues.

Explanation: A retail sales dataset — `sales_fact.csv` and `products.csv` — was loaded and inspected, revealing 61 raw transactions with 1 missing `quantity`, 1 missing `unit_price`, 2 missing `sales_amount`, 1 duplicate row, and inconsistent `region` text casing (`north`, `south` alongside proper-cased values). Every Power Query transformation from the Week 5 Power BI phase was replicated in Pandas to keep both tools consistent.

Python Code:

```
# Setup & Load
import pandas as pd
import numpy as np

sales = pd.read_csv("sales_fact.csv")
products = pd.read_csv("products.csv")

sales.head()

# Inspect
print("Shape:", sales.shape)
print("Columns:", sales.columns.tolist())
print("Data Types:\n", sales.dtypes)
print("Missing values:\n", sales.isnull().sum())
print("Duplicate rows:", sales.duplicated().sum())
print("Unique region values:", sales["region"].unique())

# Clean
df_clean = df.copy()

# 1. Remove unnecessary column
df = df.drop(columns=["notes"])

# 2. Fix data types
df["order_date"] = pd.to_datetime(df["order_date"])

# 3. Standardize inconsistent text in 'region'
df["region"] = df["region"].str.strip().str.title()

# 4. Handle missing values
df["quantity"] = df["quantity"].fillna(5)
df["unit_price"] = df["unit_price"].fillna(1999)

# 5. Fix incorrect value (negative quantity -> positive)
df.loc[df["quantity"] < 0, "quantity"] = df.loc[df["quantity"] < 0, "quantity"].abs()
df["quantity"] = df["quantity"].astype(int)

# 6. Recompute sales_amount cleanly
df["sales_amount"] = df["quantity"] * df["unit_price"]

# 7. Remove duplicate records
df = df.drop_duplicates()

print("Shape after cleaning:", df.shape)
print("Missing values after cleaning:\n", df.isnull().sum())
print("Duplicate rows after cleaning:", df.duplicated().sum())
```

📷 Raw Dataset: BEFORE CLEANING

	sale_id	order_date	product_id	region	payment_method	quantity	unit_price	sales_amount	notes
0	5001	2024-06-01	P05	West	Debit Card	9.0	3299.0	29691.0	NaN
1	5002	2024-06-03	P10	North	Debit Card	4.0	14999.0	59996.0	NaN
2	5003	2024-06-05	P07	West	UPI	8.0	6999.0	55992.0	NaN
3	5004	2024-06-07	P04	South	Debit Card	8.0	1999.0	15992.0	NaN
4	5005	2024-06-09	P04	West	UPI	5.0	NaN	NaN	NaN

Key Findings: Raw dataset: **61 transactions**, with 4 distinct data-quality issues (nulls, a duplicate, and inconsistent text).

📷 Cleaned Dataset: AFTER CLEANING

	sale_id	order_date	product_id	region	payment_method	quantity	unit_price	sales_amount	product_name	category	unit_cost	total_cost	profit
0	5001	2024-06-01	P05	West	Debit Card	9	3299.0	29691.0	Webcam HD	Electronics	2200	19800	9891.0
1	5002	2024-06-03	P10	North	Debit Card	4	14999.0	59996.0	Gaming Chair	Furniture	8900	35600	24396.0
2	5003	2024-06-05	P07	West	UPI	8	6999.0	55992.0	Noise Cancelling Headphones	Electronics	4200	33600	22392.0
3	5004	2024-06-07	P04	South	Debit Card	8	1999.0	15992.0	Laptop Stand	Accessories	1100	8800	7192.0
4	5005	2024-06-09	P04	West	UPI	5	1999.0	9995.0	Laptop Stand	Accessories	1100	5500	4495.0

Key Findings: Cleaned & merged dataset: **60 transactions, 13 columns, 0 missing values, 0 duplicates**, exported as `sales_data_cleaned.csv` — verified as the exact dataset later used in both the EDA notebook and the Power BI dashboard.

Task 4: Exploratory Data Analysis (EDA)

Objective: Understand the statistical structure of the cleaned dataset — central tendency, spread, correlation, and anomalies — before visualizing it.

Explanation : Descriptive statistics summarize typical transaction size; correlation analysis identifies which variables actually drive revenue; and IQR-based outlier detection distinguishes genuine high-value orders from data-entry errors.

Mean sales amount:

Python Code :

```
df["order_date"] = pd.to_datetime(df["order_date"], errors="coerce")
df["month"] = df["order_date"].dt.strftime("%b")

print(df[["quantity", "unit_price", "sales_amount", "profit"]].describe())
```

Output:

Descriptive Statistics:

	quantity	unit_price	sales_amount	profit
count	60.000000	60.000000	60.000000	60.000000
mean	4.483333	6039.000000	25030.516667	10103.016667
std	2.758602	4553.828711	24582.516448	9766.505637
min	1.000000	799.000000	799.000000	349.000000
25%	2.000000	1999.000000	6998.750000	3055.500000
50%	4.500000	5249.000000	16743.500000	7641.500000
75%	7.000000	8499.000000	35246.250000	14246.500000
max	9.000000	14999.000000	104993.000000	42693.000000

Key Findings: ₹25,030.52 per transaction (median ₹16,743.50) — the mean sitting well above the median indicates a right-skewed distribution.

Correlation:

Python Code :

```
correlation_matrix = df[["quantity", "unit_price", "sales_amount", "profit"]].corr()
print(correlation_matrix)
```

Output:

	quantity	unit_price	sales_amount	profit
quantity	1.000000	-0.165495	0.473542	0.476659
unit_price	-0.165495	1.000000	0.656382	0.654595
sales_amount	0.473542	0.656382	1.000000	0.997871
profit	0.476659	0.654595	0.997871	1.000000

Key Findings: Sales Amount and Profit are near-perfectly correlated (**0.998**). Unit Price correlates more strongly with Sales (**0.656**) than Quantity does (**0.474**) — pricier items, not bigger orders, drive revenue variation.

Outliers:

Python Code :

```
Q1 = df["sales_amount"].quantile(0.25)
Q3 = df["sales_amount"].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["sales_amount"] < lower_bound) | (df["sales_amount"] > upper_bound)]
print("Upper bound:", upper_bound)
print(outliers[["sale_id", "product_name", "quantity", "sales_amount"]])
```

Output:

Upper bound: 77617.5

	sale_id	product_name	quantity	sales_amount
7	5008	Gaming Chair	7	104993.0
10	5011	Gaming Chair	6	89994.0

Key Findings: Two Gaming Chair transactions were flagged — sale_id 5008 (₹1,04,993) and sale_id 5011 (₹89,994) — both exceeding the IQR upper bound of ₹77,617.50. Both were retained as legitimate bulk orders, not data errors.

Task 5: Data Visualization

Objective: Communicate the dataset's key patterns visually using 8 distinct charts across Matplotlib and Seaborn.

Explanation: Each chart targets a different business question — trend over time, regional/category/payment comparisons, distribution shape, and variable relationships.

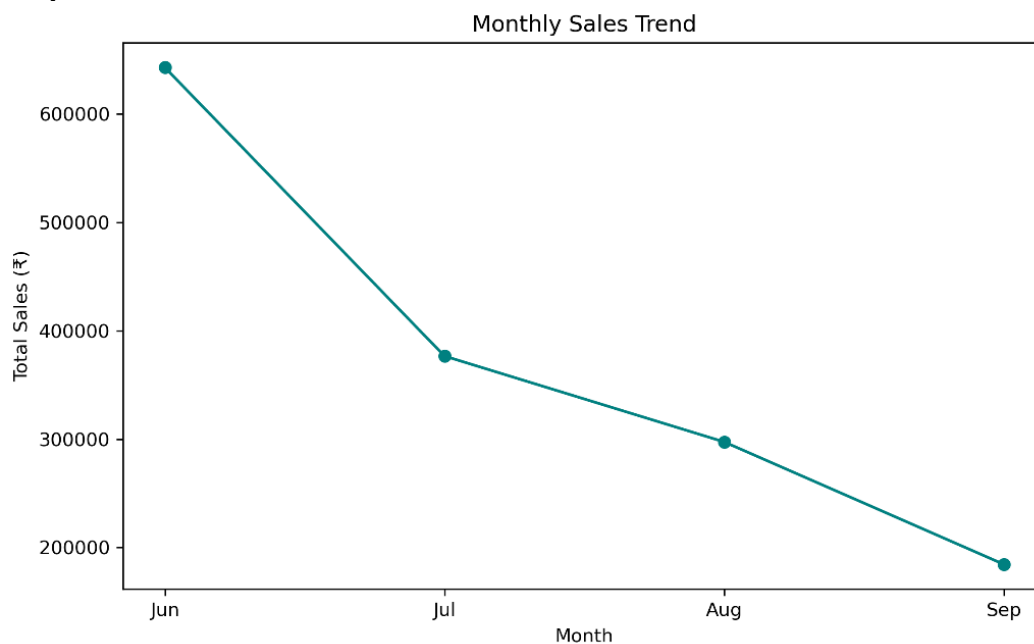
1. Monthly Sales Trend (Matplotlib — Line Chart)

Python Code :

```
month_order = ["Jun", "Jul", "Aug", "Sep"]
monthly_sales = df.groupby("month")["sales_amount"].sum().reindex(month_order)

plt.figure(figsize=(8,5))
plt.plot(monthly_sales.index, monthly_sales.values, marker="o", color="teal")
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales (₹)")
plt.tight_layout()
plt.show()
```

Output:



Explanation: Shows a steep, consistent revenue decline from June through September — the most important trend in the dataset.

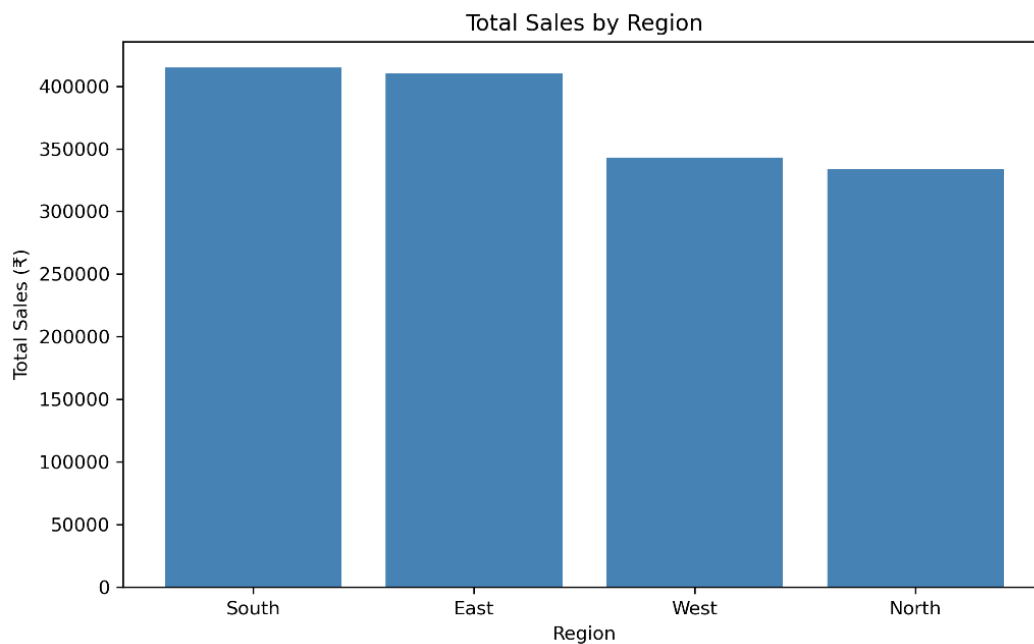
2. Total Sales by Region (Matplotlib — Bar Chart)

Python Code :

```
region_sales = df.groupby("region")["sales_amount"].sum().sort_values(ascending=False)

plt.figure(figsize=(8,5))
plt.bar(region_sales.index, region_sales.values, color="steelblue")
plt.title("Total Sales by Region")
plt.xlabel("Region")
plt.ylabel("Total Sales (₹)")
plt.tight_layout()
plt.show()
```

Output:



Explanation: South and East lead regional performance; North trails all four regions.

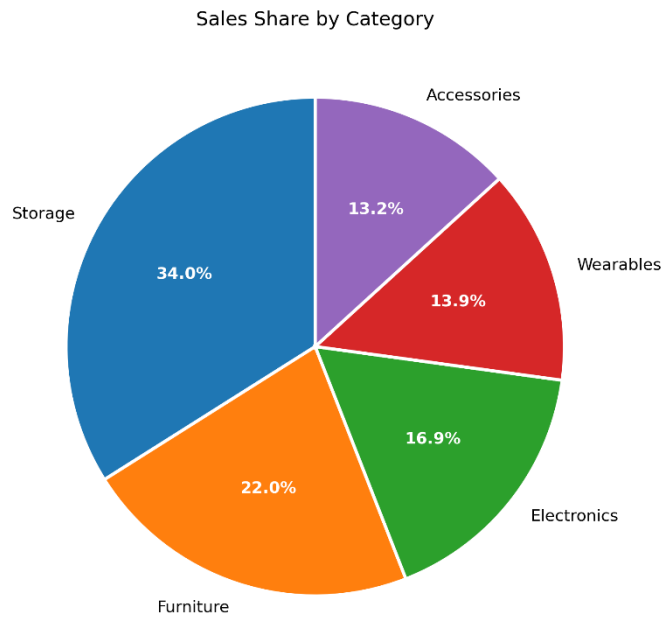
3. Sales Share by Category (Matplotlib — Pie Chart)

Python Code :

```
category_sales = df.groupby("category")["sales_amount"].sum().sort_values(ascending=False)

plt.figure(figsize=(6,6))
wedges, texts, autotexts = plt.pie(
    category_sales.values,
    labels=category_sales.index,
    autopct="%1.1f%%",
    startangle=90,
    wedgeprops={"edgecolor": "white", "linewidth": 2}
)
for autotext in autotexts:
    autotext.set_color("white")
    autotext.set_fontweight("bold")
plt.title("Sales Share by Category")
plt.tight_layout()
plt.show()
```


Output:



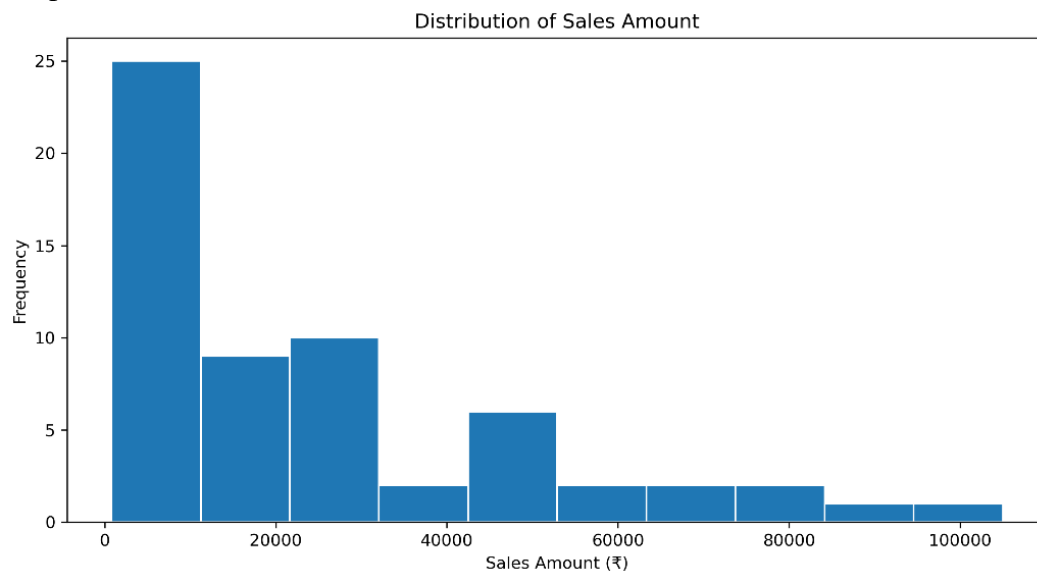
Explanation: Storage dominates at 34% of total revenue, more than double Wearables or Accessories individually.

4. Distribution of Sales Amount (Matplotlib — Histogram)

Python Code :

```
plt.figure(figsize=(9, 5))
plt.hist(df["sales_amount"], bins=10, edgecolor="white")
plt.title("Distribution of Sales Amount")
plt.xlabel("Sales Amount (₹)")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

Output:



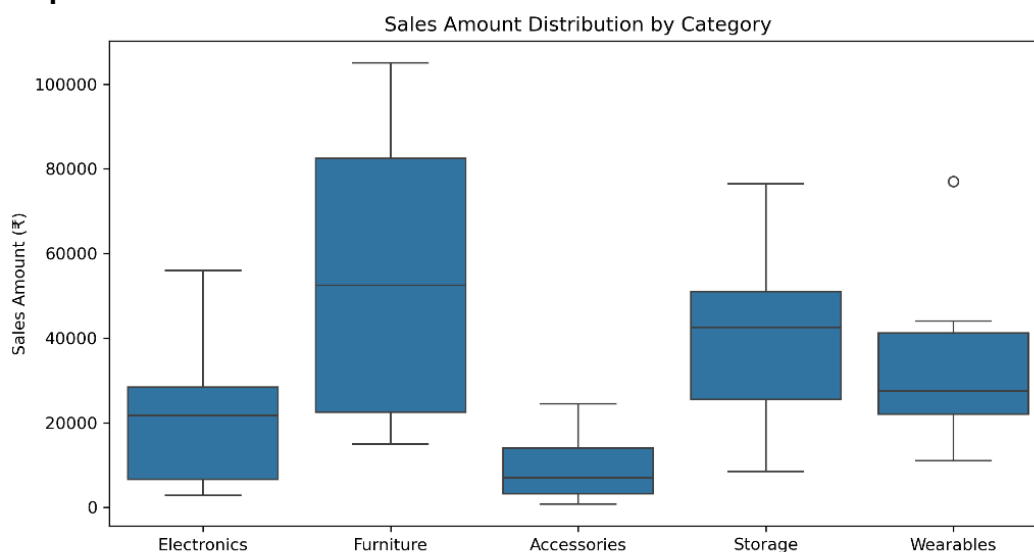
Explanation: Most transaction values cluster in the lower-to-mid range (25th–75th percentile: ₹6,998.75–₹35,246.25), with a long right tail from a handful of high-value bulk orders — visually confirming the mean (₹25,030.52) sitting above the median (₹16,743.50).

5. Sales Amount Distribution by Category (Seaborn — Box Plot)

Python Code :

```
plt.figure(figsize=(9, 5))
sns.boxplot(data=df, x="category", y="sales_amount")
plt.title("Sales Amount Distribution by Category")
plt.xlabel(" ")
plt.ylabel("Sales Amount (₹)")
plt.tight_layout()
plt.show()
```

Output:



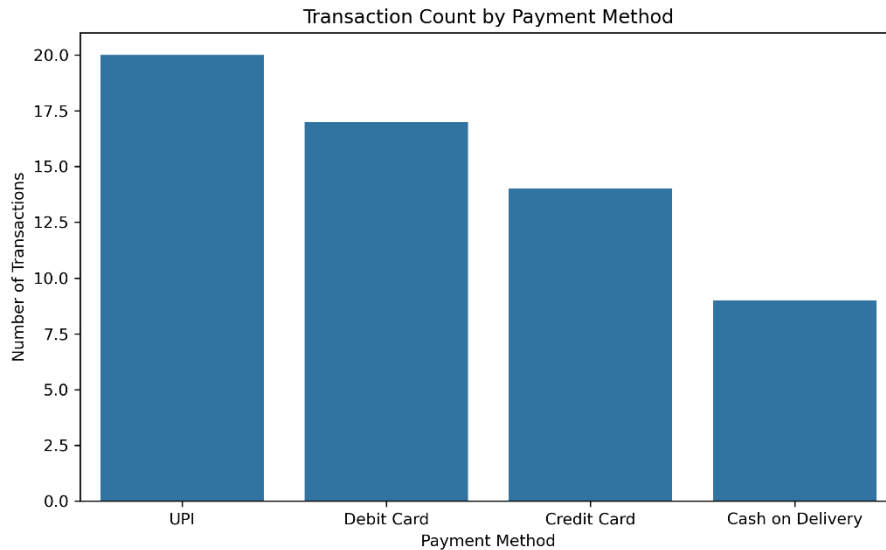
Explanation: Furniture shows the widest spread and the highest single transaction (₹1,04,993 — the bulk Gaming Chair order) despite ranking second in total revenue, while Accessories clusters tightly at the low end (median ₹6,998).

6. Transaction Count by Payment Method (Seaborn — Count Plot)

Python Code :

```
plt.figure(figsize=(8,5))
sns.countplot(data=df, x="payment_method", order=df["payment_method"].value_counts().index)
plt.title("Transaction Count by Payment Method")
plt.xlabel("Payment Method")
plt.ylabel("Number of Transactions")
plt.tight_layout()
plt.show()
```

Output:



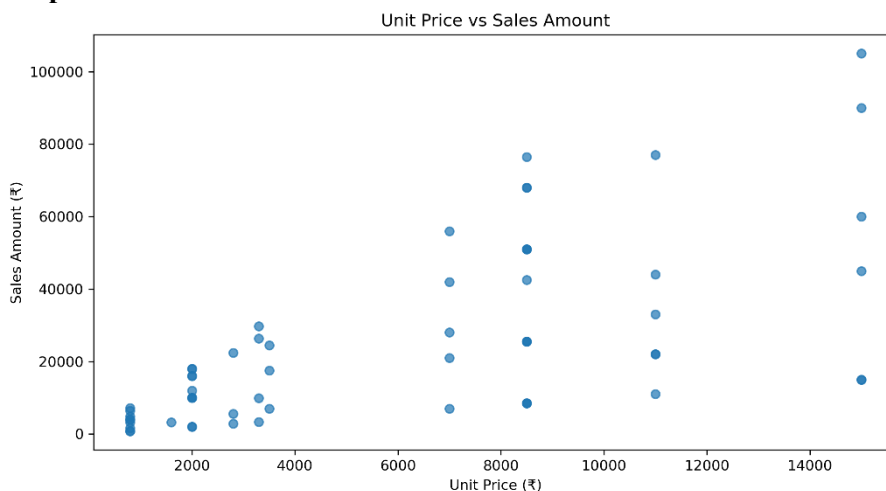
Explanation: Shows transaction volume per payment channel, complementing the revenue-based view on the Power BI dashboard.

7. Unit Price vs Sales Amount (Matplotlib — Scatter Plot)

Python Code :

```
plt.figure(figsize=(9, 5))
plt.scatter(df["unit_price"], df["sales_amount"], alpha=0.7)
plt.title("Unit Price vs Sales Amount")
plt.xlabel("Unit Price (₹)")
plt.ylabel("Sales Amount (₹)")
plt.tight_layout()
plt.show()
```

Output:



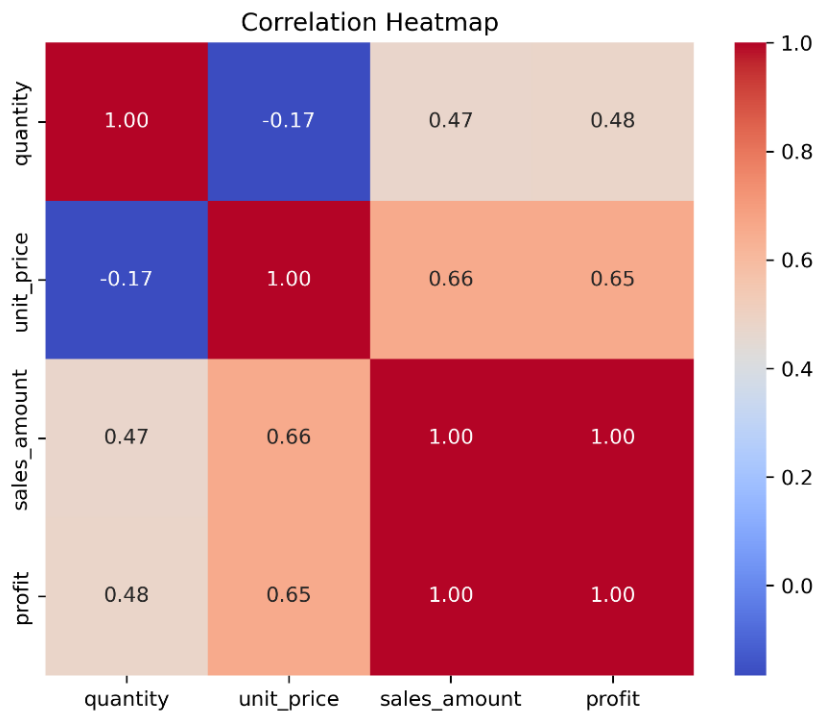
Explanation: Visualizes the moderate positive relationship ($r = 0.656$) between price and revenue per transaction, confirming price is a stronger revenue driver than quantity ($r = 0.474$).

8. Correlation Heatmap (Seaborn — Heatmap)

Python Code :

```
plt.figure(figsize=(6,5))
sns.heatmap(df[["quantity", "unit_price", "sales_amount", "profit"]].corr(),
            annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```

Output:



Explanation: Visually confirms sales_amount and profit move almost in lockstep (0.998), and highlights unit_price as a stronger revenue driver than quantity alone.

Key Findings:

- Revenue is concentrated in fewer, higher-value transactions rather than high order volume.
 - Furniture and the two Gaming Chair sales are the primary sources of outlier-level variance in the dataset.
-

Task 6: Power BI Dashboard

Objective: Build an interactive Business Intelligence dashboard that translates the Python EDA findings into a self-service executive view.

Explanation: The dashboard uses `sales_data_cleaned` (the identical 60-row, 13-column dataset exported from the Python pipeline), ensuring the two tools produce matching top-line figures. Four core KPIs are layered with Month-over-Month (MoM) measures for trend context, across five visuals covering time, geography, category, payment channel, and a cross-tabulated detail view.

Code/DAX — Core KPI Measures:

Dax

```
Total Sales Amount = SUM(sales_data_cleaned[Sales Amount])
Total Profit = SUM(sales_data_cleaned[Profit])
Total Transactions = COUNTROWS(sales_data_cleaned)
Profit Margin = DIVIDE([Total Profit], [Total Sales Amount], 0)
```

Code/DAX — Month-over-Month Measure (representative example):

Dax

```
Sales MoM % =
VAR CurrentMonth = SELECTEDVALUE('sales_data_cleaned'[Month])
VAR CurrentSales = [Total Sales Amount]
VAR PreviousSales =
    SWITCH(
        CurrentMonth,
        "Jul", CALCULATE([Total Sales Amount], REMOVEFILTERS('sales_data_cleaned'[Month]),
'sales_data_cleaned'[Month] = "Jun"),
        "Aug", CALCULATE([Total Sales Amount], REMOVEFILTERS('sales_data_cleaned'[Month]),
'sales_data_cleaned'[Month] = "Jul"),
        "Sep", CALCULATE([Total Sales Amount], REMOVEFILTERS('sales_data_cleaned'[Month]),
'sales_data_cleaned'[Month] = "Aug"),
        BLANK()
    )
RETURN
IF(CurrentMonth = "Jun", BLANK(),
    IF(ISBLANK(PreviousSales), BLANK(), DIVIDE(CurrentSales - PreviousSales, PreviousSales)))
```

Explanation: Uses `SWITCH` to explicitly map each month to its predecessor (the dataset is a fixed 4-month window, not a continuous date table), and `REMOVEFILTERS` + `CALCULATE` to correctly retrieve the prior month's total without interference from the current filter context. The same pattern was replicated for Profit, Transactions, and Profit Margin MoM measures.

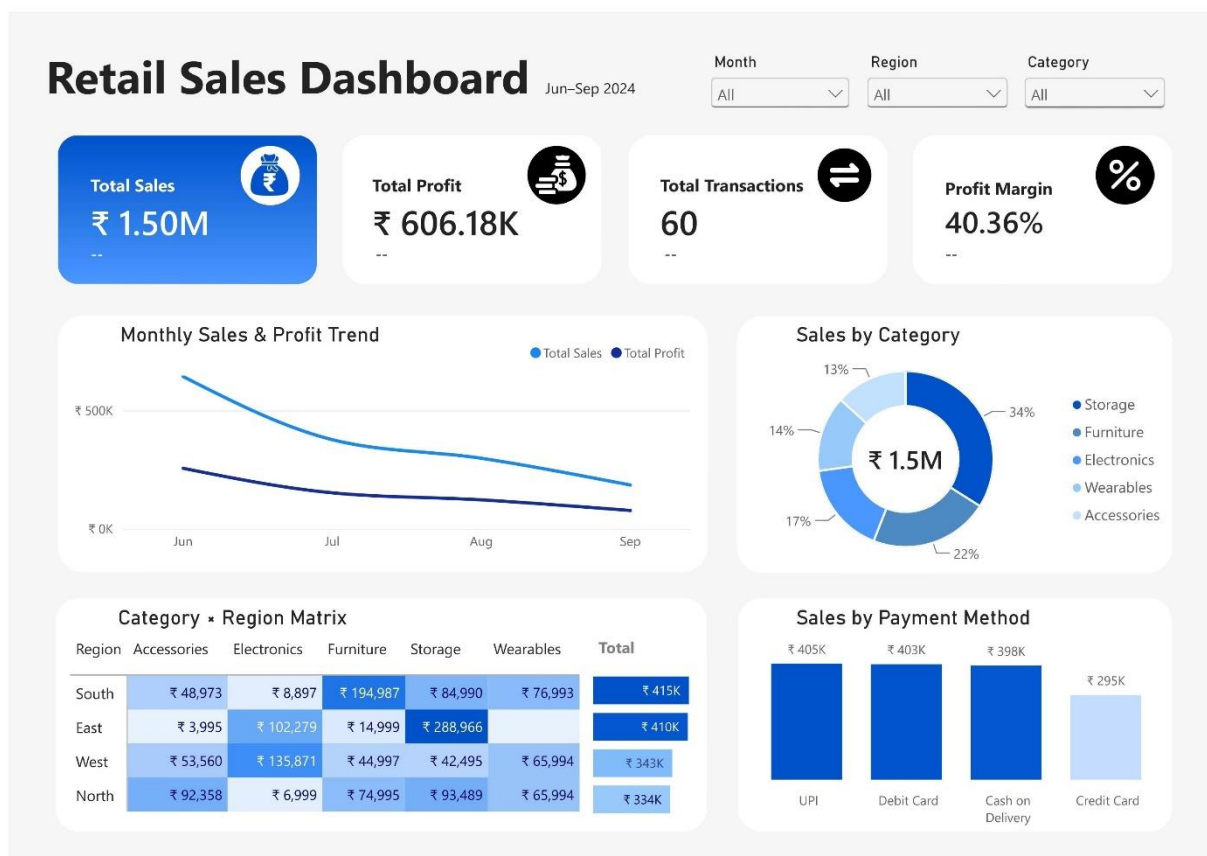
Dashboard KPI Results:

KPI	Result
Total Sales	₹1.50M
Total Profit	₹606.18K
Total Transactions	60
Profit Margin	40.36%

Visualizations :

1. **Monthly Sales & Profit Trend** — Line chart, X: Month, Y: Total Sales & Total Profit.
2. **Sales by Category** — Donut chart, Legend: Category, Values: Total Sales.
3. **Category × Region Matrix** — Rows: Category, Columns: Region, Values: Total Sales.
4. **Sales by Payment Method** — Column chart, X: Payment Method, Y: Total Sales.

Slicers: Month, Region, Category — enabling dynamic, self-service filtering across all visuals.



Key Findings:

- Python and Power BI totals matched exactly (₹15,01,831 sales / ₹6,06,181 profit), confirming cross-tool validation between the notebook and the dashboard.
- With the Month slicer defaulted to "All," the MoM KPI sub-labels correctly display "-" (blank) since `SELECTEDVALUE` cannot resolve a single month — this is expected DAX behavior, not an error.

Task 7: Business Insights & Recommendations

Objective: Translate the combined Python EDA and Power BI dashboard findings into actionable business conclusions.

Key Insights

1. **Severe revenue decline across the analysis period** — sales fell from ₹6,43,016 in June to ₹1,84,544 in September, a drop of roughly 71% peak-to-trough.
2. **South (₹4,14,840) and East (₹4,10,239) are the dominant regions**, together contributing over half of total revenue, while North (₹3,33,835) lags behind.
3. **Storage is the top-performing category**, contributing approximately 34% of total revenue (₹5,09,940) — more than any other single category.
4. **Credit Card payments lag significantly behind UPI, Debit Card, and Cash on Delivery** (₹2,95,451 vs. ₹3,98K–₹4,05K for the other three).
5. **High-value items drive revenue more than high-volume orders** — Unit Price correlates more strongly with Sales (0.656) than Quantity does (0.474), meaning pricing and product mix matter more than order size.

Business Recommendations

Recommendation 1 — Investigate the severe Q3 sales slump and launch a targeted Q4 retention campaign.

The consistent month-over-month decline (confirmed by both the EDA groupby analysis and the dashboard's MoM measures) is the single most urgent signal in the data. Action: cross-reference the decline against marketing activity, stock availability, and seasonality to isolate the cause, then design a Q4 campaign targeting the regions/categories showing the steepest drop-off.

Recommendation 2 — Concentrate inventory and marketing spend on Storage products in the South and East regions.

Storage is the top revenue category overall, and South/East are the top two regions — aligning inventory priority and promotional spend with this proven combination is lower-risk than spreading resources evenly.

Task 8: GitHub Documentation & Project Presentation

Objective: Package the entire project into a professional, portfolio-ready GitHub repository.

Explanation: The README follows a standard data analytics portfolio structure so any recruiter or reviewer can understand the project's purpose, methodology, and findings without opening the underlying files.

Code — README.md Structure :

Markdown

```
# 🏪 Retail Sales Performance Analytics

**End-to-end analytics case study** – analyzing 60 cleaned retail transactions to
diagnose a sharp Q3 revenue decline, identify the strongest regions and product
categories, and translate the findings into actionable Q4 business recommendations
using Python and Power BI.

</p>

<p align="center">

! [Python](https://img.shields.io/badge/Python-3776AB?style=for-the-
badge&logo=python&logoColor=white)
! [Pandas](https://img.shields.io/badge/Pandas-150458?style=for-the-
badge&logo=pandas&logoColor=white)
! [NumPy](https://img.shields.io/badge/NumPy-013243?style=for-the-
badge&logo=numpy&logoColor=white)
! [Matplotlib](https://img.shields.io/badge/Matplotlib-11557C?style=for-the-
badge&logo=matplotlib&logoColor=white)
! [Seaborn](https://img.shields.io/badge/Seaborn-4C72B0?style=for-the-
badge&logo=python&logoColor=white)
! [Power BI](https://img.shields.io/badge/Power_BI-F2C811?style=for-the-
badge&logo=powerbi&logoColor=black)
! [Jupyter](https://img.shields.io/badge/Jupyter-F37626?style=for-the-
badge&logo=jupyter&logoColor=white)
! [Git](https://img.shields.io/badge/Git-F05032?style=for-the-
badge&logo=git&logoColor=white)
! [GitHub](https://img.shields.io/badge/GitHub-181717?style=for-the-
badge&logo=github&logoColor=white)

</p>

---

# 👤 Author

**Brojo Mohan Dutta**

**InternNova Data Analytics Internship**

**Week 6 Assignment: Git, GitHub & Final Data Analytics Project**

---
```


🚀 Project Overview

This end-to-end data analytics project analyzes retail sales transactions from **June 2024 to September 2024** to understand a sharp decline in revenue and identify the key factors that should influence Q4 business decisions.

Using **Python, Pandas, NumPy, Matplotlib, Seaborn, Power BI, DAX, and Git/GitHub**, the project transforms raw transactional data into reliable analytical data, explores sales patterns and relationships, and communicates the findings through an interactive business dashboard.

The project follows the **Ask → Prepare → Process → Analyze → Share → Act** framework, taking the analysis from raw CSV files through data cleaning and EDA to Power BI reporting and business recommendations.

🎯 Business Problem

The retail business experienced a significant slowdown in revenue throughout Q3 2024.

The primary business question is:

> **Why is retail revenue declining from June to September 2024, where is the revenue coming from, and what actions should the business take to stabilize performance in Q4?**

The analysis focuses on:

- Monthly revenue trends
- Regional performance
- Product category performance
- Payment method performance
- Transaction-level outliers
- Price versus quantity as revenue drivers
- Actionable Q4 business opportunities

📊 Project Summary

Category	Details
Project	Retail Sales Performance Analytics
Author	Brojo Mohan Dutta
Course	Google Data Analytics Professional Certificate
Case Study	Capstone Project 2
Dataset	Retail Transactional Sales Dataset
Raw Transactions	61
Clean Transactions	60
Time Period	June–September 2024
Product Categories	5
Regions	4
Payment Methods	4
Total Sales	₹1.50M
Total Profit	₹606.18K
Profit Margin	40.36%

```
| Tools | Python, Pandas, NumPy, Matplotlib, Seaborn, Power BI, DAX, Jupyter,
Git/GitHub |
| Dashboard | Interactive Power BI Dashboard |
| Project Type | End-to-End Data Analytics |
```

🛠 Technology Stack

```
| Tool | Purpose |
|-----|-----|
| Python | Data preparation, transformation and analytical workflow |
| Pandas | Data cleaning, manipulation and aggregation |
| NumPy | Numerical analysis and data processing |
| Matplotlib | Exploratory data visualization |
| Seaborn | Statistical visualization and correlation analysis |
| Jupyter Notebook | Interactive Python analysis and documentation |
| Power BI | Interactive dashboard and business reporting |
| DAX | KPI and Month-over-Month calculations |
| Git | Version control |
| GitHub | Repository management and project documentation |
```

🔄 Project Workflow

```
<p align="center">

</p>
```

The project followed this analytical workflow:

1. Load raw retail transaction and product datasets
2. Inspect data structure, data types, missing values and duplicates
3. Clean and standardize the transactional data using Python
4. Merge transaction data with the product dimension
5. Calculate sales, cost and profit metrics
6. Perform exploratory data analysis using Python
7. Identify trends, correlations and transaction-level outliers
8. Build an interactive Power BI dashboard
9. Validate Power BI results against Python outputs
10. Translate findings into business recommendations

📁 Dataset Overview

The project uses two primary source datasets:

```
### `sales_fact.csv`
```

Raw transaction-level sales records containing:

- Sale ID
- Order Date
- Product ID
- Region
- Payment Method

- Quantity
- Unit Price
- Sales Amount
- Notes

`products.csv`

Product dimension table containing:

- Product ID
- Product Name
- Category
- Unit Cost

The two datasets were combined during the data preparation stage to create the final analysis-ready dataset.

The raw dataset contained **61 transactions**, while the final cleaned dataset contained **60 transactions across 13 columns**.

📊 Exploratory Data Analysis

The EDA phase was performed using **Pandas, NumPy, Matplotlib and Seaborn**.

Analysis includes:

- Descriptive Statistics
- Monthly Sales Analysis
- Regional Sales Analysis
- Category Sales Analysis
- Payment Method Analysis
- Correlation Analysis
- Sales Distribution Analysis
- IQR-Based Outlier Detection
- Unit Price vs Sales Analysis
- Category-Level Sales Distribution
- Transaction Count Analysis
- Correlation Heatmap

📈 Dashboard Preview

Retail Sales Dashboard

![Retail Sales Dashboard](04_power_bi/Retail_Sales_Dashboard.jpg)

Live Dashboard: `🔗 INSERT LIVE POWER BI DASHBOARD LINK HERE`

The Power BI dashboard provides an executive view of:

- Total Sales
- Total Profit
- Total Transactions
- Profit Margin
- Monthly Sales & Profit Trend

- Sales by Category
- Category × Region Matrix
- Sales by Payment Method

Interactive slicers are available for:

- Month
- Region
- Category

📊 Key Findings

- Revenue declined consistently every month, falling from **₹643,016** in June to **₹184,544** in September, representing an approximately **71% peak-to-trough decline**.
- **South and East** were the strongest regions, generating approximately **₹825K** or **55% of total sales** combined.
- **Storage** was the top-performing category, contributing approximately **34% of total revenue (₹509,940)**.
- **Credit Card** sales were significantly lower at **₹295,451**, compared with approximately **₹398K–₹405K** across UPI, Debit Card and Cash on Delivery.
- **Unit Price** had a stronger correlation with Sales Amount (**0.656**) than Quantity (**0.474**), suggesting that pricing and product mix were more influential revenue drivers than order size in this dataset.
- Sales Amount and Profit showed an almost perfect correlation of **0.998**, reflecting the strong relationship between revenue and profitability.
- Two high-value **Gaming Chair** transactions exceeded the IQR upper bound, but were retained because they represented legitimate bulk orders rather than data errors.
- The final dataset contained **0 missing values and 0 duplicates**, providing a validated foundation for EDA and Power BI reporting.

💡 Business Recommendations

📈 Recover the Revenue Trend

Launch a targeted Q4 retention and revenue-recovery initiative.

First determine whether the decline is primarily driven by transaction volume, average order value, inventory availability, marketing activity, or seasonality before allocating significant campaign budget.

📈 Double Down on Proven Strength

Prioritize **Storage products in the South and East regions** for inventory availability, targeted promotions and marketing investment.

This focuses resources on the strongest existing revenue combination rather than distributing resources evenly across all regions and categories.

🛠️ Fix the Credit Card Gap

Investigate the significant performance gap in Credit Card transactions.

Audit potential issues involving:

- Payment failures
- Authorization problems
- Checkout friction
- Processing issues
- Customer payment preferences

A targeted improvement could help recover additional revenue if the gap is caused by payment or checkout friction.

📁 Repository Structure

```text

Retail-Sales-Analysis/

|— README.md  
|— LICENSE

|— 01\_datasets  
| |— products.csv  
| |— sales\_data\_cleaned.csv  
| |— sales\_fact.csv

|— 02\_python\_cleaning  
| |— Dataset\_Selection&Data\_Preparation.ipynb  
| |— Dataset\_Selection&Data\_Preparation.py

|— 03\_python\_eda&visualization  
| |— eda\_visualizations  
| |— EDA&Visualizations.ipynb  
| |— Exploratory\_Data\_Analysis\_&Visualizations.py

|— 04\_power\_bi  
| |— Retail\_Sales\_Performance\_Dashboard.pbix  
| |— Retail\_Sales\_Dashboard.jpg

|— 05\_documentation  
| |— Retail\_Sales\_Performance\_Analytics\_Presentation.pptx  
| |— Retail\_Sales\_Performance\_Analytics\_Presentation.pdf  
| |— Retail\_Sales\_Performance\_Analysis\_Report.docx  
| |— Retail\_Sales\_Performance\_Analysis\_Report.pdf  
| |— Workflow.png

```

Repository Structure

```
Retail-Sales-Analysis/  
  
|— README.md  
|— LICENSE  
  
|— 01_datasets  
|   |— products.csv  
|   |— sales_data_cleaned.csv  
|   └─ sales_fact.csv  
|  
|— 02_python_cleaning  
|   |— Dataset_Selection&Data_Preparation.ipynb  
|   └─ Dataset_Selection&Data_Preparation.py  
|  
|— 03_python_eda&visualization  
|   |— eda_visualizations  
|   |— EDA&Visualizations.ipynb  
|   └─ Exploratory_Data_Analysis_&Visualizations.py  
|  
|— 04_power_bi  
|   |— Retail_Sales_Performance_Dashboard.pbix  
|   └─ Retail_Sales_Dashboard.jpg  
|  
|— 05_documentation  
|   |— Retail_Sales_Performance_Analytics_Presentation.pptx  
|   |— Retail_Sales_Performance_Analytics_Presentation.pdf  
|   |— Retail_Sales_Performance_Analysis_Report.docx  
|   └─ Retail_Sales_Performance_Analysis_Report.pdf
```

Key Findings:

- The repository structure separates raw data, processing notebooks, and final deliverables — matching standard data analytics portfolio conventions and making the project immediately reviewable by a recruiter or grader.